CS4780 - Lab 5: R Literate Programming

Contents

Introduction 1
- Literate programming . . . 1
- Literate programming in R . . . 2

R Markdown 3
- Markdown syntax . . . 3

Exercise 1 3

Literatre programming with R Markdown 4
- The strucuture of a R Markdown file . . . 4
- Code 'chunks' . . . 6
- Workflow in RStudio . . . 7

Exercise 2 7
- The YAML header . . . 8
- R Markdown tips and tricks . . . 9

Other programming languages 10

Exercise 3 10

Further information 11

Submitting your Lab assignment

Add the answers for the individual exercises to `Lab05_Worksheet.txt`. At the end of the lab, please submit the filled worksheet as **plain text** via the corresponding link on Blackboard. *No other formats will be considered.*
After submission deadline, the answers to the exercises will be published as Lab05_Solutions.txt on Blackboard.

Introduction

In this tutorial, we will learn how to write reports in R. More specifically, we will learn how to write documents that contain both, textual descriptions and executable data analysis. The combination of these two elements is what is called literate programming.

Literate programming

Wikipedia (https://en.wikipedia.org/wiki/Literate_programming) describes literate programming as "a programming paradigm introduced by Donald Knuth in which a computer program is given an explanation of its logic in a natural language, such as English, interspersed with snippets of macros and traditional source code, from which compilable source code can be generated."

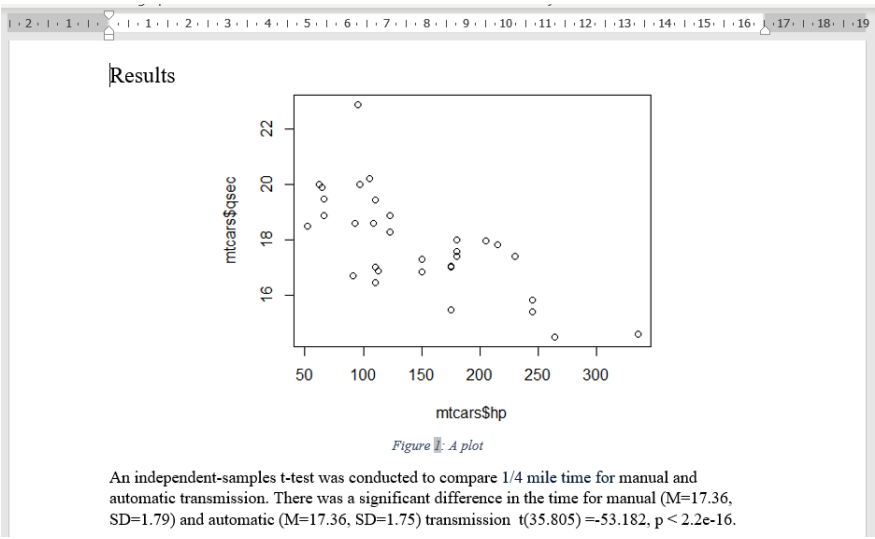
The literate programming approach should help to understand and replicate your results, find errors and suggest enhancements.

Traditionally, we write code that analysis our data and creates plots. The results of the analysis and the plots are then copied over to a document and described independently of the procedure that produced the output.

For example, we have the below code in R

```
plot(mtcars$hp, mtcars$qsec)        # Make a scatter plot
t.test(mtcars$am, mtcars$qsec)      # run a t-test
```

We now create a separate document (e.g. in Word) where we report the results of the analysis and show the plot:



Clearly, this is both, an inefficient and error-prone way to analyse data and report results; every time we run a new analysis we need to update the document, we can lose the script file, and we can (and will) make mistakes in copy information from one place to the other.

These are precisely the issues *literate programming* in R tries to tackle: One common source for doing the analysis *and* documenting the results.

Literate programming in R

Literate programming in R is provided through the build-in 'Sweave' function and the 'knitr' package (https://cran.r-project.org/web/packages/knitr/index.html).

We will use the knitr package that adds many new capabilities to Sweave and is also fully supported by RStudio (If the knitr package is not installed, do install it using `install.packages("knitr")`)

A literate programming document in R combines sections written in a text markup language (LaTeX, Markdown, HTML) with 'chunks' or R code. We then run the document through ta filter program that converts the markup text to formatted text, executes the chunks of R code, and inserts the output of the chunks into the formatted document.

One of the advantages of literate programming in R is the multitude of supported output formats; we can create HTML, LaTeX, pdf, Word, and other formats directly from one single original document. Have a look here for a gallery of different supported formats.

R Markdown

The knitr package supports several document markup languages for literate programming. Here we will learn how to use the Markdown syntax (https://daringfireball.net/projects/markdown/). Specifically, we will learn how to do literate programming in R where we mix descriptions written in Markdown with snippets of functioning R code.

Using R Markdown we can also directly publish a report, create websites, etc. This is beyond the scope of this tutorial. For more in-depth information have a look at https://annakrystalli.me/rrresearch/05_literate-prog.html

Markdown syntax

In the words of its creator, John Gruber "Markdown is a text-to-HTML conversion tool for web writers. Markdown allows you to write using an easy-to-read, easy-to-write plain text format[...]."

Below a list of some of the most used Markdown tags.

Formatting Text in `*italics*`, in `**bold**` and `` `teletype` ``.

Images You can include an image using `![Wally](wally.jpg)`.

Code style `This text will appear as code`

Headers

```
# This is an H1
## This is an H2
###### This is an H6
```

Lists

Unordered list:

```
* Red
* Green
* Blue
```

Ordered list:

```
1. Red
1. Green
1. Blue
```

Mathematical expressions

Supports mathematical notations through MathJax. You can write LaTeX math expressions inside a pair of dollar signs, e.g. `$\alpha+\beta$` renders $\alpha + \beta$. You can use the display style with double dollar signs:

```
$$\bar{X}=\frac{1}{n}\sum_{i=1}^nX_i$$
```

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

Exercise 1

Head over to the StackEdit online markdown editor at https://stackedit.io

Your task is to reproduce the output show below

Submit the Markdown code as solution to this exercise.

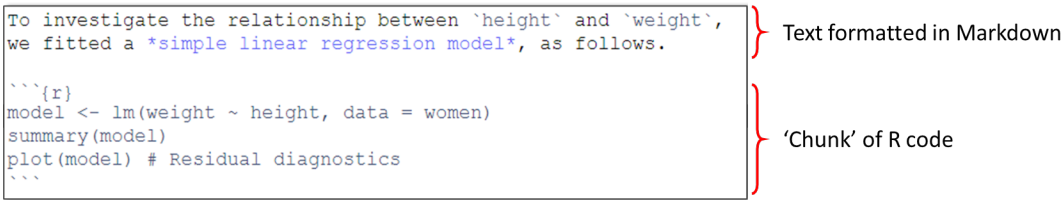
Literatre programming with R Markdown

The strucuture of a R Markdown file

A single R Markdown ('.Rmd') file integrates

- documentantion in Markdown (.md) and
- code in the R programming language (R)

Below an example of snippet of an R Markdown file:



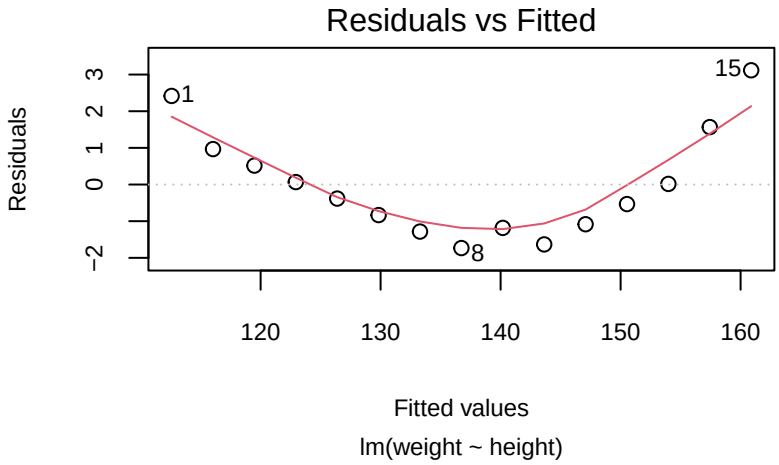
To create the output in the chosen format, we have to render the RMarkdown file using the 'knit' command. This will produce the document shows below:

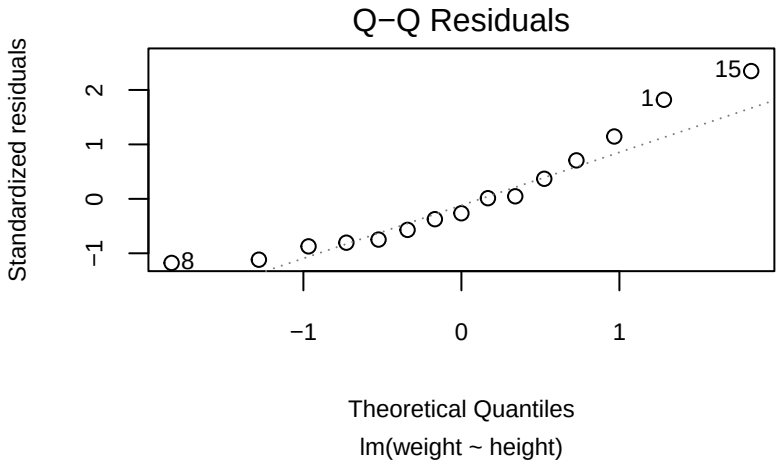
To investigate the relationship between `height` and `weight`, we fitted a *simple linear regression model*, as follows.

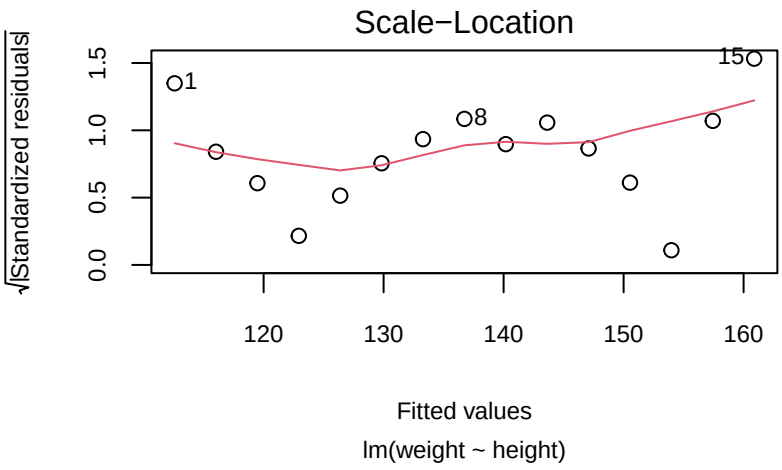
```
model <- lm(weight ~ height, data = women)
summary(model)
```

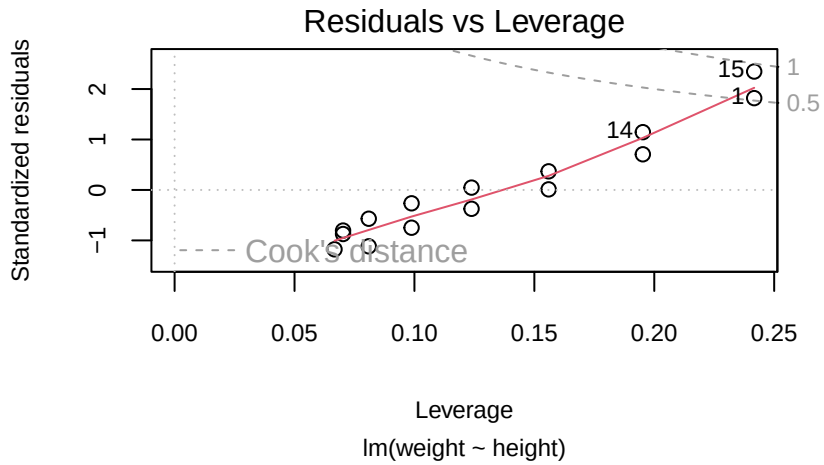
```
##
## Call:
## lm(formula = weight ~ height, data = women)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7333 -1.1333 -0.3833  0.7417  3.1167
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -87.51667    5.93694  -14.74 1.71e-09 ***
## height        3.45000    0.09114   37.85 1.09e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.525 on 13 degrees of freedom
## Multiple R-squared:  0.991,  Adjusted R-squared:  0.9903
## F-statistic:  1433 on 1 and 13 DF,  p-value: 1.091e-14
```

```r
plot(model, cex.lab=.75, cex.axis=.75, cex.main=3, cex.sub=.75)
```









As we can see, this is quite an efficient way to keep the description and the analysis output in a single place.

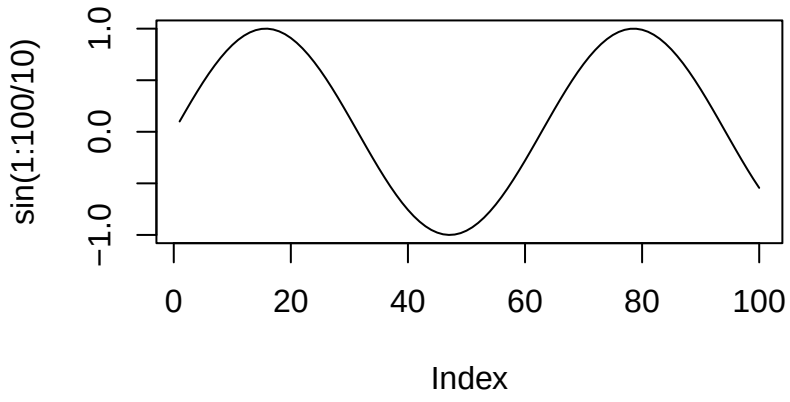
Code 'chunks'

In R Markdown lingo, pieces of R code that are embedded in the markup document are called 'chunks'. In the rendering process, chunks for R code are executed and the output is inserted into the document. The processing the code and the generation of output is controlled via chunk options.

Code chunks are sections enclosed between "'{r} and "'. Note that you can give the chunk a name to make identification easier and re-use it later.

```
```{r ChunkName}
plot(sin(1:100/10), type='l')</br>
```
```

will produce the following plot:



Note for R Studio users: You can quickly insert a code chunk with:

- the keyboard shortcut **`Ctrl + Alt + I`** (**OS X: `Cmd + Option + I`**)
- the Add Chunk  command in the RStudio toolbar

Evaluate r code inline

You can also evaluate code inline with the text, i.e. not in a separate block.

For example, the R Markdown statement The sine of 0.5 is ‘r sin(0.5)‘. Will produce the following output: “The sine of 0.5 is 0.4794255”

Chunk options

Various options allow for a fine-grain control of the output of the chunks. Below a list of the most common options (default value in brackets). A comprehensive list can be found here: https://yihui.name/knitr/options/

- `eval` (TRUE): If FALSE, knitr will not run the code in the code chunk
- `echo` (TRUE): If FALSE, knitr will not display the code in the code chunk above its results in the final document.
- `error` (TRUE): If FALSE, knitr will not display any error messages generated by the code.
- `message` (TRUE): If FALSE, knitr will not display any messages generated by the code.
- `warning` (TRUE): If FALSE, knitr will not display any warning messages generated by the code

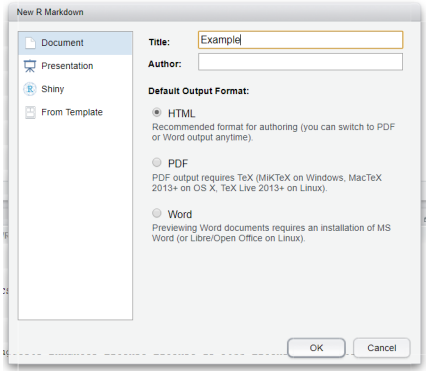
Note for R Studio users: Once you have inserted a chunk, you can use the cogwheel symbol on the top right of the chunk to control chunk options.

Workflow in RStudio

RStudio has excellent support for working literate programming in R, especially support for R Markdown.

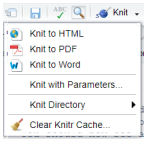
Exercise 2

- Create a new R Markdown file in RStudio: `File->New File->R Markdown`

- Use the value as shown in here: 

- You should now see a document stub like this

```
---
title: "Example"
output: html_document
---
```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```
R Markdown
```

- Save the file as `example.Rmd`
- Render the document to your format of choice by clicking on the 'knit' button 

- Alternatively, you can run the command `rmarkdown::render(input = "example.Rmd")`

Note: This exercise does not need to be submitted.

The YAML header

What you see at the very top of the document is the YAML (yet another markup language) header. This piece of special code lets you specify additional options before rendering your document. In RStudio, changing the knit output options will change the YAML header.

```
---
title: "Title of the document"
author: "Ulysses Bernardet"
date: "9 December 2021"
output: pdf_document
---
```

Many of the document output options such as the target format can be changed via the `Output Options` command accessible from the cogwheel symbol. Here some additional HTML options that are not accessible in this way:

Floating table of contents

You can add a table of contents (TOC) using the toc option and specify a floating toc using the `toc_float option`. For example:

```
---
output:
```

```
  html_document:
    toc: true
    toc_float: true
---
```

Themes

We can use predefined themes to control the appearance of HTML documents. Themes drawn from the Bootswatch theme library include `default`, `cerulean`, `journal`, `flatly`, `darkly`, `readable`, `spacelab`, `united`, `cosmo`, `lumen`, `paper`, `sandstone`, `simplex`, and `yeti`.

```
---
output:
  html_document:
    theme: cosmo
---
```

R Markdown tips and tricks

Formatting tables

By default, R Markdown does not display tables very nicely:

```
head(airquality)
```

```
##   Ozone Solar.R Wind Temp Month Day
## 1    41     190  7.4   67     5   1
## 2    36     118  8.0   72     5   2
## 3    12     149 12.6   74     5   3
## 4    18     313 11.5   62     5   4
## 5    NA      NA 14.3   56     5   5
## 6    28      NA 14.9   66     5   6
```

There are several commands for formatting tables more nicely, for examle `knitr::kable()`:

```
library(knitr)
kable(head(airquality), caption = "New York Air Quality Measurements")
```

Table 1: New York Air Quality Measurements

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8.0	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
NA	NA	14.3	56	5	5
28	NA	14.9	66	5	6

Even better in combination with the package 'kableExtra)':

```
library(knitr)
library(kableExtra)
kable(head(airquality), caption = "New York Air Quality Measurements") %>%
                         kable_styling(bootstrap_options = "bordered", full_width = F)
```

Table 2: New York Air Quality Measurements

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8.0	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
NA	NA	14.3	56	5	5
28	NA	14.9	66	5	6

`DT::datatable()` creates interactive html tables:

```
library(DT)
datatable(airquality, caption = "New York Air Quality Measurements")
```

Other programming languages

We can not only include code chuncks in R, but in a number of other languages (provided they are installed). For example, we can include pieces of Python with

````
```{python}
x = ['To', 'be', 'or', 'not', 'to', 'be']
y = [i.upper() for i in x]
print(" ".join(y) + 5 * '?!')
```
````

Which will be produce the output below:

```
## TO BE OR NOT TO BE?!?!?!?!?!
```

The command `names(knitr::knit_engines$get())` will give you a list of available language engines. For further information, check https://bookdown.org/yihui/rmarkdown/language-engines.html for a list of supported language.

Exercise 3

Reproduce the below output in R Studio using R Markdown. Do not forget to include the YAML header.

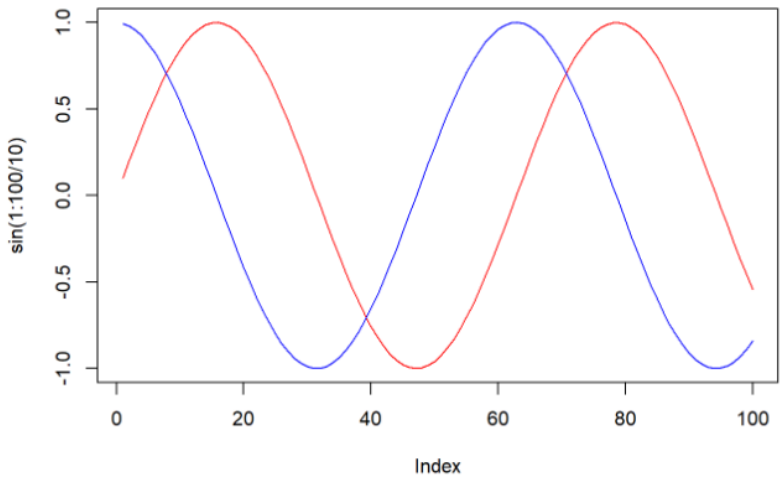
Literate Programming Exercise

- Plotting

Plotting

A plot of sine and cosine.

```
plot(sin(1:100/10), type = 'l', col="red")
lines(cos(1:100/10), col="blue")
```



Further information

- R Markdown http://rmarkdown.rstudio.com
- knitr http://yihui.name/knitr
- R Markdown reference guide and cheat sheet https://www.rstudio.com/resources/cheatsheets/